Risk Management & Audit Implementation

Project: Python-Based Automated Web Penetration Testing Framework
Assessment Environment: Local cybersecurity testing laboratory
Assessment Date: November 8, 2026
Document Type: Risk Management and Security Audit Documentation

---

1. Executive Summary

The project incorporates a risk management and security audit program designed to identify, assess, prioritize, validate, and document cybersecurity risks within a controlled testing environment.

The program combines vulnerability assessment, automated security testing, evidence collection, security control validation, and structured reporting. Its purpose is to demonstrate how an organization can establish a repeatable process for evaluating security risks and determining whether existing security controls are functioning effectively.

The implementation uses a Python-based automated web penetration-testing framework to assess web applications for potential security weaknesses. Identified findings are categorized according to severity, confidence, and validation status. Supporting evidence is collected and maintained through an evidence management system that records timestamps, evidence identifiers, investigator information, and SHA-256 hashes.

The assessment methodology is informed by established cybersecurity practices, including NIST risk management principles, NIST security assessment guidance, and OWASP application security recommendations.

The project demonstrates practical risk assessment and security auditing capabilities within a laboratory environment. Enterprise-wide deployment and continuous compliance monitoring remain potential future enhancements.

2. Risk Management Framework

2.1 Risk Management Objectives

The risk management program establishes procedures for:

1. Identifying information security risks and vulnerabilities.

2. Evaluating the likelihood and potential impact of identified threats.
3. Assigning risk scores and prioritizing remediation activities.
4. Validating vulnerabilities through controlled security testing.
5. Documenting risk treatment recommendations.
6. Maintaining evidence supporting assessment conclusions.
7. Reassessing risks following remediation or changes to the environment.

The objective is to ensure that security weaknesses are evaluated based on their potential consequences rather than simply the number of vulnerabilities discovered.

2.2 Asset and Threat Identification

The assessment focuses on identifying assets and components that could introduce security risks.

Assets within the laboratory environment include:

Asset	Description	Potential Security Concern
Web application	OWASP Juice Shop	Application vulnerabilities and unauthorized access
Web endpoints	Discovered application routes and API endpoints	Improper input validation and access control
HTTP security headers	Application response headers	Browser-based security weaknesses
Authentication functionality	Login and POST request handling	Authentication weaknesses
Python testing framework	Custom automated security assessment toolkit	Detection errors and incomplete testing
Evidence repository	Evidence files and integrity records	Evidence modification or loss
Testing environment	Local Docker-based laboratory	Misconfiguration and unintended exposure

Threat scenarios considered include:

- SQL injection and other injection attacks.
- External redirects and unsafe URL handling.
- Missing or improperly configured security headers.
- Unauthorized access to application functionality.

- Exposure of sensitive information.
- Improper handling of authentication requests.
- Evidence tampering or loss of audit records.

2.3 Risk Quantification Methodology

A numerical risk-scoring methodology was established to provide a consistent method for prioritizing identified risks.

The project uses a 5 × 5 risk assessment model.

Risk Score = Likelihood × Impact

Both variables are rated from 1 to 5.

Likelihood assessment

Score	Rating	Description
1	Rare	Highly unlikely under normal conditions
2	Unlikely	Possible but requires unusual circumstances
3	Possible	Could reasonably occur
4	Likely	Expected to occur under relevant conditions
5	Almost Certain	Highly probable or repeatedly observed

Impact assessment

Score	Rating	Description
1	Negligible	Minimal operational or security consequences
2	Minor	Limited disruption or exposure
3	Moderate	Meaningful impact on application security or operations
4	Major	Significant compromise or operational disruption

5	Severe	Critical compromise, extensive data exposure, or major business impact

Risk classification

Risk Score	Risk Level	Recommended Response
1–4	Low	Document and monitor
5–9	Moderate	Schedule corrective action
10–16	High	Prioritize remediation
17–25	Critical	Immediate attention and risk treatment

Risk scores are assigned using the documented likelihood and impact assessments. They represent the project's risk methodology, not automatically calculated scores for every scanner finding.

2.4 Risk Treatment Procedures

Each identified risk should receive one of four treatment decisions:

- **Mitigate:** Implement security controls or corrective changes to reduce risk.
- **Accept:** Formally acknowledge the remaining risk and document the justification.
- **Transfer:** Assign some financial or operational consequences to another party.
- **Avoid:** Eliminate the activity or configuration creating the risk.

Following remediation, the affected asset should be retested to determine whether the vulnerability has been eliminated or whether residual risk remains.

2.5 Inherent and Residual Risk

The assessment distinguishes between:

- **Inherent risk:** Risk before considering existing security controls.
- **Residual risk:** Risk remaining after applicable controls have been implemented.

The project uses retesting and evidence comparison to evaluate whether corrective actions reduce the original exposure.

---

3. Security Audit Program

3.1 Audit Program Objectives

The security audit program provides a repeatable methodology for evaluating the security posture of the testing environment and its applications.

The program establishes procedures for:

- Defining the scope of an assessment.
- Identifying systems and application components.
- Reviewing existing security configurations.
- Performing authorized vulnerability assessments.
- Validating security weaknesses.
- Collecting and preserving supporting evidence.
- Documenting findings and remediation recommendations.
- Producing structured audit reports.

3.2 Audit Scope and Authorization

All technical assessments are conducted against authorized laboratory systems.

The primary web application assessment target is OWASP Juice Shop running within the local Docker environment.

Before testing begins, the assessment methodology identifies:

Requirement	Procedure
Scope	Identify authorized applications, endpoints, and systems
Objectives	Define the security controls and vulnerabilities being evaluated
Authorization	Confirm permission to perform testing
Testing boundaries	Establish prohibited systems and activities
Timeline	Define testing and reporting periods
Deliverables	Identify required findings, evidence, and reports

Testing activities are restricted to the approved laboratory environment.

3.3 Security Audit Procedures

The audit process follows a structured sequence.

Phase 1 – Planning

- Define assessment objectives.
- Identify assets and testing targets.
- Establish authorized testing boundaries.
- Identify relevant security requirements.
- Determine evidence collection procedures.

Phase 2 – Information Gathering

- Identify accessible application pages.
- Crawl application routes and forms.
- Discover API endpoints and parameters.
- Review HTTP response behavior.
- Identify existing security configurations.

Phase 3 – Security Testing

- Perform automated security checks.
- Evaluate HTTP security headers.
- Test application input handling.
- Examine redirect behavior.
- Review authentication-related functionality.
- Identify potential vulnerabilities.

Phase 4 – Validation

- Review automated findings.
- Compare expected and observed application behavior.
- Perform additional tests to reduce false positives.
- Assign confidence and validation status.
- Compare findings with independent security assessment results where available.

Phase 5 – Evidence Collection

- Assign unique evidence identifiers.
- Record collection timestamps.
- Associate evidence with test and finding identifiers.
- Calculate SHA-256 hashes.
- Maintain evidence integrity and verification records.

Phase 6 – Reporting

- Summarize assessment results.
- Categorize findings by severity.
- Explain potential business and technical impacts.
- Provide remediation recommendations.
- Identify unresolved risks and testing limitations.

---

4. Compliance Frameworks and Security Control Mapping

The project uses recognized cybersecurity frameworks as references for organizing security requirements and evaluating control effectiveness.

These mappings demonstrate alignment with selected control objectives; they do not represent formal organizational certification or a completed enterprise compliance audit.

4.1 NIST Cybersecurity Framework 2.0

The project maps its activities to the six functions of NIST CSF 2.0.

NIST Function	Project Implementation
Govern	Document security responsibilities, testing authorization, and assessment procedures
Identify	Identify application assets, endpoints, and potential security risks
Protect	Evaluate security configurations and application protection mechanisms
Detect	Use automated testing to identify potential vulnerabilities
Respond	Document findings and recommend corrective actions
Recover	Establish retesting and verification procedures following remediation

4.2 NIST SP 800-53

Selected NIST SP 800-53 control families provide additional guidance.

Control Family	Project Application
RA – Risk Assessment	Risk identification, scoring, and prioritization

CA – Assessment, Authorization, and Monitoring	Security assessments and validation procedures
SI – System and Information Integrity	Vulnerability identification and security testing
AU – Audit and Accountability	Evidence registers, timestamps, and verification records
CM – Configuration Management	Review of application security configurations
AC – Access Control	Assessment of authentication and access-related weaknesses

4.3 OWASP Security Recommendations

OWASP guidance is used to evaluate application-level security.

Relevant areas include:

- Injection vulnerabilities.
- Security misconfiguration.
- Authentication weaknesses.
- Identification and authorization failures.
- Security logging and monitoring.
- Unsafe application redirects and input handling.

The automated framework incorporates selected application security checks, with additional coverage identified as future development work.

---

5. Advanced Security Control Validation

5.1 Validation Methodology

Security control validation determines whether identified controls are functioning as intended.

The project uses a combination of automated testing, manual result review, and independent tool comparison.

Each test should document:

1. Test identifier.
2. Target application or endpoint.
3. Security control being evaluated.

4. Testing procedure.
5. Expected result.
6. Actual result.
7. Supporting evidence.
8. Validation status.
9. Recommended corrective action.

5.2 Implemented Control Testing

The following tests have been performed in the laboratory environment.

Test	Security Control	Testing Method	Observed Result
HTTP security headers	Browser security protections	Inspect HTTP responses	Missing CSP identified on tested responses
External redirect	Safe URL handling	Submit redirect test payload	External HTTP 302 redirect observed
SQL injection	Input validation	Compare baseline and conditional responses	Results inconclusive
Application discovery	Asset and endpoint visibility	Automated crawling	Pages, scripts, and endpoints discovered
Evidence integrity	Evidence preservation	SHA-256 comparison	Matching hashes verified in transfer test

5.3 Independent Validation

OWASP ZAP was used as an additional assessment tool to compare results from the custom Python framework.

The comparison helps identify:

- Findings that are detected by both tools.
- Findings requiring further investigation.
- Potential false positives.
- Security weaknesses that may not be detected by the custom framework.

Independent validation is particularly important because automated scanners can identify suspicious behavior without conclusively establishing exploitability.

5.4 Finding Validation Standards

The custom framework uses four validation statuses:

Status	Meaning
Confirmed	Sufficient evidence establishes the vulnerability
Potential	Evidence suggests a vulnerability but requires further validation
Inconclusive	Testing did not produce sufficient evidence for a reliable conclusion
Not Detected	The test did not identify the vulnerability

Findings also receive a confidence rating and severity classification.

This prevents uncertain results from being presented as confirmed security vulnerabilities.

---

6. Audit Findings and Risk Recommendations

6.1 Identified Assessment Results

The following examples demonstrate the use of the risk assessment and audit methodology.

Finding 1 – Missing Content Security Policy

- Finding ID: V-003
- Category: Security Misconfiguration
- Severity: Low
- Validation: Potential
- Evidence: HTTP response header inspection and OWASP ZAP results.
- Observation: The tested application responses did not consistently provide a Content-Security-Policy header.
- Risk: The absence of an appropriate CSP can reduce browser-side protection against certain content injection attacks.
- Recommendation: Develop and test an appropriate CSP based on the application's legitimate resource requirements. Validate the policy before production deployment.

Finding 2 – External Redirect Behavior

- Finding ID: V-002
- Category: Improper Redirect Handling
- Severity: Medium
- Validation: Potential
- Evidence: Redirect test demonstrating an HTTP 302 response with an external Location destination.

- Observation: The application accepted a test value that resulted in a redirect to an external destination.
- Risk: Improperly validated redirects may enable phishing or other user-redirection attacks.
- Recommendation: Validate redirect destinations against an approved allowlist and reject untrusted external destinations.

Finding 3 – SQL Injection Testing

- Finding ID: V-001
- Category: Injection Testing
- Validation: Inconclusive
- Evidence: Baseline and conditional response comparisons and malformed input testing.
- Observation: The malformed input test produced an HTTP 500 response. However, true-condition and false-condition response counts were identical at 56, so the conditional comparison did not establish SQL injection.
- Risk: An underlying input-handling weakness cannot be ruled out based on the available testing.
- Recommendation: Investigate server-side error handling, review query construction, use parameterized queries, and perform additional controlled validation.

These results demonstrate why findings must be evaluated according to evidence and confidence rather than automatically treated as confirmed vulnerabilities.

6.2 Risk Prioritization

The audit program prioritizes corrective actions according to:

- Severity of the finding.
- Likelihood of exploitation.
- Potential operational and business impact.
- Confidence in the available evidence.
- Exposure of the affected application component.
- Availability of compensating security controls.

A scanner severity rating is not automatically equivalent to the project's calculated enterprise risk score. Likelihood and impact must be separately assessed before assigning that score.

---

7. Evidence Management and Audit Accountability

7.1 Evidence Collection

A dedicated evidence management system was implemented to support the audit process.

Evidence records include:

- Evidence_ID
- Timestamp
- Test_ID
- Finding_ID
- Evidence_Type
- Target
- Description
- Filename
- SHA256
- Collected_By
- Validation_Status
- Notes

The evidence collection process associates individual test results with their corresponding security findings.

7.2 Evidence Integrity

The project uses SHA-256 hashing to verify that collected evidence has not been altered.

The evidence management system includes:

- An evidence register.
- A chain-of-custody log.
- A verification log.
- Unique evidence identifiers.
- Evidence-to-finding relationships.

A file transfer verification test was performed using `GET-001_request.txt`.

The original and transferred copies produced matching SHA-256 hashes, demonstrating that the file contents remained consistent during the test.

7.3 Audit Trail

The chain-of-custody records provide accountability by documenting who collected evidence, when it was collected, and how it is associated with the assessment.

This provides a foundation for repeatable audits and subsequent investigation.

---

8. Professional Audit Reporting Standards

The project uses a structured reporting format designed to communicate technical findings to both technical personnel and management.

Each security audit report should contain:

1. **Executive Summary:** Overall security assessment and significant risks.
2. **Assessment Scope:** Systems, applications, and testing boundaries.
3. **Methodology:** Tools, procedures, and validation techniques.
4. **Risk Assessment:** Risk scoring and prioritization methodology.
5. **Findings:** Unique identifiers, severity, confidence, and validation status.
6. **Supporting Evidence:** Relevant logs, screenshots, response data, and evidence references.
7. **Business Impact:** Potential consequences of each finding.
8. **Recommendations:** Corrective actions and suggested controls.
9. **Limitations:** Testing constraints and areas not assessed.
10. **Conclusion:** Overall assessment and recommended next steps.

Reports should distinguish confirmed vulnerabilities from potential and inconclusive findings.

The reporting methodology is informed by NIST SP 800-115 security testing and assessment guidance.

---

9. Performance Metrics and Continuous Improvement

The program establishes measurable indicators to evaluate the effectiveness of its assessment and audit processes.

Metric	Purpose
Assets identified	Measures asset discovery coverage
Endpoints discovered	Measures application discovery
Security tests executed	Measures testing activity
Findings identified	Measures discovered security issues
Confirmed findings	Measures validated vulnerabilities
Potential findings	Tracks issues requiring additional validation
False-positive rate	Evaluates detection accuracy
Evidence verification rate	Measures evidence integrity

Remediation completion rate	Measures corrective action progress
Retest success rate	Evaluates the effectiveness of remediation

Metrics should be calculated from actual scan records, test results, and audit evidence. No organization-wide performance figures are claimed by this laboratory implementation.

9.1 Continuous Improvement

The audit program follows a recurring improvement cycle:

1. Assess the environment.
2. Identify and prioritize risks.
3. Implement corrective actions.
4. Retest affected components.
5. Compare results.
6. Update risk records and documentation.
7. Improve testing procedures.

This creates a repeatable approach to managing vulnerabilities and validating security controls.

---

10. Implementation Status and Limitations

The project has established several working components supporting the risk management and audit program.

Requirement	Implementation Status	Supporting Evidence
Risk assessment methodology	Documented	5 × 5 risk scoring methodology
Threat and asset identification	Partially implemented	Juice Shop and endpoint discovery
Risk treatment procedures	Documented	Mitigation, acceptance, transfer, and avoidance
Security audit procedures	Implemented at laboratory scope	Testing methodology and framework workflow
Compliance framework mapping	Documented	NIST CSF, NIST SP 800-53, OWASP

Automated security testing	Implemented	Python penetration-testing toolkit
Security control validation	Partially implemented	Header, redirect, and SQL injection tests
Independent validation	Implemented	OWASP ZAP comparison
Evidence management	Implemented	Evidence register and chain-of-custody system
Evidence integrity validation	Tested	SHA-256 verification
Professional audit reporting	Implemented at project scope	Structured HTML security report
Enterprise-wide continuous monitoring	Not implemented	Future enhancement
Formal compliance certification	Not implemented	Outside current project scope

The project operates within a controlled laboratory environment. It demonstrates selected enterprise risk management and security audit practices but does not claim to provide a complete enterprise governance, risk, and compliance platform.

---

11. Future Enhancements

The following improvements would expand the program toward a more comprehensive enterprise implementation:

1. **Centralized Risk Register:** Maintain a persistent inventory of assets, vulnerabilities, risk scores, owners, and remediation status.
2. **Automated Risk Scoring:** Calculate likelihood, impact, inherent risk, and residual risk from structured assessment records.
3. **Continuous Asset Monitoring:** Integrate additional asset discovery and network monitoring capabilities.
4. **Compliance Automation:** Map security findings to specific NIST control requirements and maintain control assessment records.
5. **Remediation Tracking:** Track corrective actions, responsible parties, deadlines, and retesting outcomes.
6. **Threat Intelligence Integration:** Use a platform such as OpenCTI to provide additional context about known threats and indicators.

7. **Dashboard and Metrics:** Present risk trends, vulnerability distribution, remediation progress, and audit performance.
8. **Scheduled Security Audits:** Establish recurring assessments to identify newly introduced vulnerabilities.

These improvements would help transform the existing laboratory assessment framework into a more centralized risk management and security audit platform.

---

12. Conclusion

The project establishes a practical foundation for enterprise-oriented risk management and security auditing through the integration of automated vulnerability assessment, structured risk classification, security control validation, evidence preservation, and professional reporting.

The custom Python framework demonstrates the ability to identify and evaluate application security weaknesses. Independent testing with OWASP ZAP provides additional validation, while the evidence management system supports accountability and integrity verification.

The documented risk quantification methodology provides a consistent approach for prioritizing security issues according to likelihood and impact. Compliance mappings to NIST and OWASP provide a structured reference for evaluating security practices and identifying areas requiring improvement.

Although enterprise-wide governance and continuous monitoring have not yet been deployed, the project demonstrates the core technical and procedural elements required to establish a repeatable security risk management and audit program.

Overall outcome: A functional laboratory-level risk assessment and security audit implementation, supported by documented procedures, automated testing, validation evidence, and structured reporting.